Library Before Proof: Making LLM-Generated Rocq Usable by Mathematicians

Guillaume Baudart [1] **Marc Lelarge** [2]

Abstract

LLM-assisted formalization now produces thousands of lines of Lean or Rocq for research-level mathematics, but the resulting code remains hard to read for a working mathematician who is not a Rocq expert. We argue that the bottleneck is no longer closing goals, but the library built *around* the proof terms. We propose a three-artifact approach: mathcomp-style Rocq backed by an explicit coding-style skill, a Lean-style blueprint cross-linking informal math to formal lemmas, and a verification PDF pairing each definition and theorem statement with its informal version. We demonstrate it on two case studies: a textbook chapter of Stanley's *Enumerative Combinatorics* and a Rocq formalization of Quasi-Borel Spaces. Both libraries contain zero `Admitted` and zero custom axioms; both were produced in a single Docker environment by Claude Opus driving Rocq through an MCP server, to which we contributed. The contribution is not the libraries themselves but a proposal for the artifacts that should accompany LLM-generated Rocq for the mathematicians, plus the tooling to produce them.

1. Introduction

Putnam 2025 Problem A5 is, on paper, a one-page combinatorics exercise: the sign sequences for which the descent pattern admits the most permutations of $\{1, \ldots, n\}$ are the two alternating patterns, and the maximum is the n-th Euler/zigzag number (Stanley, 2011; Mathematical Association of America, 2026). The Axiom Math team's autonomous Lean prover failed this problem during the 2025 competition and, on a post-hoc attempt, produced a formal proof in 518 minutes using 9.1 M tokens, totalling 2,054 lines, 52 theorems and 3,074 tactics (Axiom Math, 2025). None of these numbers reflect the *informal* difficulty of A5. They reflect that the basic enumerative combinatorics of descents and Euler numbers is not packaged as a library in either Mathlib (Lean) or mathcomp (Rocq), so a goal-closing system has to re-derive it from scratch every time.

The question we explore is therefore not *can an LLM close goals in Lean or Rocq*, which is by now well-established, but **can the resulting library be read, audited and reused by a working mathematician without proof-assistant expertise?** We treat this as an open design question and propose an answer: a Rocq-MCP server (an interface that lets the LLM drive the Rocq prover step by step) and a coding-style skill keep the LLM mathcomp-compatible from the first line, while an auto-generated blueprint and a verification PDF expose the resulting code in the mathematician's own language. We demonstrate the stack end-to-end on two libraries built with Claude Opus and ship every piece so others can extend or replace it.

This paper is about the *deliverable*, not about closing goals. Our central contribution is the **recipe** of three mathematician-facing artifacts (§3); the case studies validate it, and the tooling makes it reproducible:

The recipe (§3), our main contribution: mathcomp-style Rocq (backed by an explicit, machine-checkable style guide), a blueprint cross-linking informal mathematics to formal lemmas, and a verification PDF pairing each statement with its informal counterpart.

Two case studies (§4) that validate the recipe and show where it breaks at scale: Library E (Stanley §1.4/§1.6 — descents, Eulerian numbers, the alternating-set maximum; $\approx$22,500 lines, 889 named results, the *full* recipe) and Library Q (Quasi-Borel Spaces; 8,913 lines, 412 proofs, first two artifacts only). Both: zero `Admitted`, zero custom axioms, same Docker environment, model, and MCP tools.

Supporting tooling and a finding: five features we contributed upstream to the Rocq-MCP server (§5), and what the two studies reveal about scaling — the verification PDF is practical at Library E's scale but the blueprint must carry the load at Library Q's (§6).

[1]Université Paris Cité, Inria, CNRS [2]Inria - Ecole Normale Supérieure PSL Research University. Correspondence to: Guillaume Baudart <guillaume.baudart@inria.fr>, Marc Lelarge <marc.lelarge@inria.fr>.

Proceedings of the 43rd International Conference on Machine Learning, Seoul, South Korea. PMLR 306, 2026. Copyright 2026 by the author(s).

Table 1. Stanley *Enumerative Combinatorics* §1.3–§1.6 (cycles, descents, Euler numbers): coverage in mathcomp.

Item	Covered?
Permutations as a type, parity, $\binom{n}{k}$	✓
Polynomials `poly`, derivative, polynomial ring	✓
Integer count $\mathrm{inv}(w)$ of inversions	✗
Multinomial coefficient $\binom{n}{c_1,\ldots,c_k}$	✗
Descent set $D(w)$; $\alpha_n(S)$, $\beta_n(S)$	✗
Möbius inversion on $2^{[n-1]}$ (Eq. 1.34)	✗
MacMahon multinomial formula	✗
Eulerian polynomial / Eulerian number	✗
Major index, inv $\leftrightarrow$ maj, Foata	✗
Alternating perms, Euler number E_n	✗
Formal power series {`fps`}	✗
Euler EGF $\sum_n E_n x^n/n! = \sec x + \tan x$ (Prop. 1.6.1)	✗
Worpitzky's identity (§1.4)	✗
Stirling cycle numbers $c(n,k)$ and EGF	✗
Carlitz q-Eulerian (q-analogue of §1.4)	✗

Table 2. Quasi-Borel Spaces: coverage in mathcomp-analysis.

Item	Covered?
`measurableType`, σ-algebras, Lebesgue integral, Fubini	✓
Pushforward measure, s-finite kernels	✓
Giry monad with the three monad laws	✓
Standard distributions (Normal, Bernoulli)	✓
Cantor space (Alexandroff–Hausdorff)	✓
Cartesian closure for locally compact regular spaces	✓
Quasi-Borel space (random elements, QBS morphisms)	✗
QBS $\leftrightarrow$ measurable correspondence ($L \dashv R$)	✗
Standard Borel as a class; $\mathbb{R} \cong \mathbb{R} \times \mathbb{R}$ encoding	✗
QBS probability monad	✗
Bayesian normalizer; Bayesian regression showcase	✗

2. Background

LLM-assisted theorem proving in Rocq / Lean. Recent autonomous systems (AlphaProof on IMO 2024 (Google DeepMind, 2024), Axiom Math on Putnam 2025 (Axiom Math, 2025)) show that LLMs can close goals at olympiad and graduate-research levels. We focus on what happens *after* the goal is closed: the produced code is typically a single-purpose proof script that does not survive as a library.

Mathcomp coding style and the blueprint tradition. Mathematical Components (*mathcomp*) (The Mathematical Components team, 2020) and its analysis extension (The MathComp-Analysis team, 2024) form the large, uniformly-styled Rocq library that working mathematicians read and extend: terse code following strict naming and proof conventions (the SSReflect idiom). The Lean community has, separately, developed the *blueprint* pattern (used by Liquid Tensor Experiment, Sphere Eversion, FLT, and many others (Massot, 2024)): a hyperlinked document that states the mathematics informally and cross-links every informal statement to its formal counterpart. A Rocq port of this tooling (`rocqblueprint`, a `plasTeX` plugin) is available independently (Cirpons, 2024); we adopt it for both libraries, and add a third, finer-grained artifact where feasible.

What is in mathcomp, what is not. Tables 1 and 2 summarize the gap our two libraries fill. "✗" means there is no named definition or lemma in the library. Both were surveyed against a switch pinning Rocq 9.1.1, mathcomp 2.5.0, mathcomp-analysis 1.16.0, and mathcomp-finmap 2.2.2;

3. The recipe: three mathematician-facing artifacts

The core of the paper is a methodology built from three artifacts, each answering a different question for a mathematician who is not a Rocq expert. Mathcomp-style Rocq backed by a style guide answers *"can I read the proof at all?"*; the blueprint answers *"what proves what, and how do the results depend?"*; and the verification PDF answers *"is this really the statement I mean?"* The first two scale to any size; the verification PDF stays practical only for a few dozen headline results — the central trade-off we return to in §4. We take the three in turn.

(a) Mathcomp-style Rocq, backed by a machine-checkable style guide. "Follow mathcomp style" is only useful as advice if it can be checked. We make it checkable through a coding-style skill (The LLM4Rocq team, 2026d) developed alongside the two libraries: core naming and proof conventions, domain-specific idiom files, a playbook of verified cleanup patterns, and a mechanical audit script for high-yield style violations (full inventory in Appendix C). The skill keeps the generated Rocq close enough to mathcomp idiom that a mathcomp-literate mathematician can inspect it without first learning a project-specific dialect.

(b) The blueprint. A hyperlinked HTML+PDF document, generated alongside the Rocq source (Cirpons, 2024), that (i) states the mathematical theorems in informal mathematical language, (ii) cross-links every informal statement to the formal Rocq lemma that proves it, and (iii) records the dependency graph between informal results. We follow the Lean blueprint convention (Massot, 2024). Both libraries ship a blueprint, included in their repositories (The LLM4Rocq team, 2026a;b).

(c) The verification PDF (Library E only). Aimed *one level lower* than the blueprint: every Rocq **definition** and every Rocq **theorem statement** appears next to its informal counterpart, so a mathematician can audit the formalization without opening Rocq. The verification PDF is where the question *"is this actually Stanley's β_n?"* gets answered. For Library E we additionally ship an auto-generated companion catalog (`lemma_catalog.tex`, $\approx$3,700 lines, 1,106 entries) listing every named result with kind, file:line, and `coqdoc` docstring. This artifact is feasible at the Eulerian scale (a few dozen headline definitions and theorems plus a mechanical catalog of the rest) but *not* at the QBS scale (with 400+ proofs), where the blueprint plays this role.

A one-page verification-PDF excerpt (Stanley's insert-max bijection, with informal statement, Rocq target, and Decoded translation) is in Appendix B.

4. Two case studies

Both libraries were produced inside the *same* reproducible environment: a Docker container with the full Rocq toolchain (Rocq, mathcomp, mathcomp-analysis, Hierarchy Builder, algebra-tactics), running an MCP server for Rocq (The LLM4Rocq team, 2026c; Anthropic, 2024) that exposes interactive proof state to the LLM, with Claude Opus given full tool permissions (file I/O, shell, the MCP tool, and the build system). Using the same infrastructure in both case studies makes the comparison cleaner: the two projects still differ in subject, scale, proof style, and mathematical domain, but not in the toolchain used to produce them.

4.1. Library E (Eulerian): full recipe

Scope. Substantial portions of Stanley *Enumerative Combinatorics* §1.4 (descents) and §1.6 (alternating permutations / Euler numbers), with the supporting machinery the headline result needed: inversions, cycles, the multinomial coefficient, the Foata correspondence, the cd-index, and the min-max tree (Stanley §1.6.3). The library's top-level theorem is `beta_alt_max` (the alternating descent set maximizes β_n). Since submission we have also closed the generating-function side: a new reusable formal-power-series sub-library (`mathcomp_fps`, 8 files) now proves Prop. 1.6.1 itself ($\sum_n E_n x^n/n! = \sec x + \tan x$, `stanley_1_6_1`, as a *formal* rather than analytic identity), Worpitzky's identity and the Eulerian OGF (§1.4), the Stirling-cycle EGF (§1.3), and Carlitz's q-analogue of §1.4. Its theorems use the standard classical axioms of `mathcomp-classical` (the same set mathcomp-analysis uses); the combinatorial core, `beta_alt_max`, and the Putnam A5 proof remain axiom-free.

Numbers. 49 Rocq files (including an 8-file reusable `mathcomp_fps` sub-library), $\approx$22,500 lines, **889 named results** (`Lemmas`/`Theorems`), 180 `Definitions`, **0 `Admitted`, 0 custom axioms** (The LLM4Rocq team, 2026a). The auto-generated catalog lists 1,106 entries, which additionally includes definitions and indented declarations inside sections.

Putnam A5, the motivating example. Library E was scoped to be the smallest library on top of which Putnam 2025 A5 (the problem on which Axiom Math's Lean prover spent 518 minutes and 2,054 lines) becomes a one-page argument. The informal derivation follows directly from §1.4 + §1.6 and a single Niven-style swap inequality (Niven, 1968); the full vignette (reformulation in Stanley's notation, symmetry, identification with the Euler number, strict inequality) is worked in Appendix A. On top of Library E, our axiom-free Rocq proof of A5 (Baudart et al., 2026) is **214 lines** across three files, i.e., roughly an order of magnitude shorter than the 2,054-line bespoke Lean development of the same statement.

Recipe. Library E exercises the full three-artifact recipe: lemma names follow Stanley's numbering (`Prop_1_4_1`, `Eq_1_34`, ...) so the verification PDF and the source line up by inspection; the blueprint mirrors the library's logical structure (descents, inversions, mmtree, omega, cd-index, propositions, see Fig. 1); the verification PDF plus the mechanically-extracted `lemma_catalog.tex` together cover the entire declared API.

4.2. Library Q (QBS): first two artifacts

Scope. Quasi-Borel Spaces (Heunen et al., 2017) on top of mathcomp-analysis: cartesian-closed category, $L \dashv R$ hom-set bijection with measurable spaces, probability monad, product measures with iterated integration, standard Borel via $\mathbb{R} \cong (0,1)$ and $\mathbb{R} \cong \mathbb{R} \times \mathbb{R}$, normal/Bernoulli/uniform distributions, and a Bayesian linear regression example. This Rocq formalization of QBS follows the existing Isabelle formalization (Hirata et al., 2021).

Numbers. 13 Rocq files, 8,913 lines, **412 proofs**, **0 `Admitted`, 0 custom axioms** (The LLM4Rocq team, 2026b).

Limitations. The formalization is not unconditional: a few results are stated in slightly weaker forms than their textbook counterparts — for instance, the probability monad's laws hold up to equivalence rather than on the nose, and the QBS/measurable correspondence is a bijection of morphism sets rather than a full natural transformation. Some of these are inherited from the Isabelle formalization (Hirata et al., 2021), some are specific to the Rocq port; all are listed in the blueprint rather than hidden in the source, and are spelled out in Appendix E.

Recipe. Library Q exercises mathcomp style and the

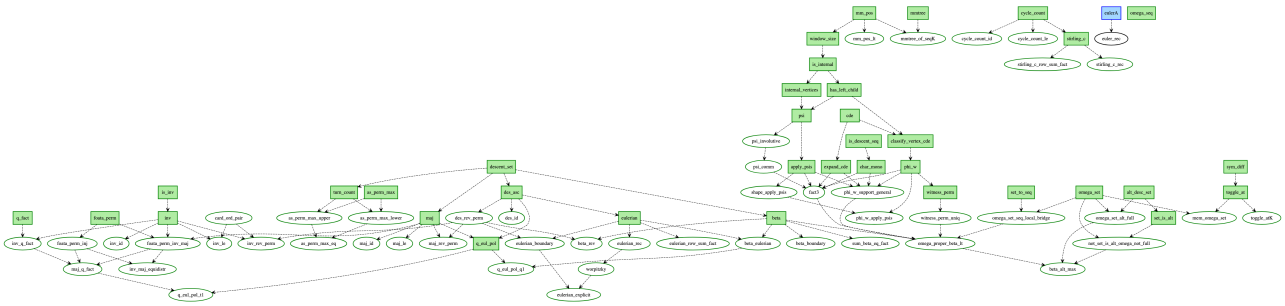


Figure 1. Dependency graph of Library E, generated automatically from the blueprint's `\uses` annotations (rectangles = definitions, ellipses = lemmas / theorems; green = proved). The graph is the blueprint's navigation surface: a mathematician can read the chapter text, click any node to jump to the formal Rocq lemma, and see at a glance which prerequisites are still missing. The headline corollary `beta_alt_max` (alternating descent set strictly maximizes β_n) sits at the bottom-right, downstream of the descents / omega machinery on the right and the Eulerian / Foata machinery on the left.

blueprint at scale, but we did not produce a verification PDF. A per-lemma audit-pairing at 400+ proofs would have been busywork; the blueprint plus the mathcomp-style source acts as the verification surface instead.

Sanity check on size. As a sanity check, we compared formal-to-informal expansion against established MathComp/Rocq developments. Both libraries land inside the human-authored band (Tab. 3): Library Q close to paper-anchored projects (`abel`), Library E close to textbook-anchored ones (`FormalPowerSeries`). This does not certify style or idiom, but it rules out the simplest failure mode: a library whose size is obviously out of family.

Project	Style	SLOC/pp	SLOC/stmt
`abel`	paper	375	159
Library Q	paper	520	173
`FormalPowerSeries`	textbook	595	320
Library E	textbook	581	245

Table 3. Coarse expansion ratios. `SLOC` = source lines of Rocq code (non-blank, non-comment). `SLOC/pp` = SLOC per page of the informal source the library is anchored to (Stanley §1.3, §1.4 and §1.6 for Library E, the LICS-2017 paper for Library Q). `SLOC/stmt` = SLOC per numbered informal statement (Definition / Lemma / Theorem / Proposition) in that source.

What the two studies show. The full three-artifact recipe is practical at Library E's scale — a few dozen headline statements plus a mechanical catalog — but not at Library Q's, where a per-lemma verification PDF over 400+ proofs would be busywork and the blueprint plus the mathcomp-style source carries the audit load instead. This is the paper's main empirical finding: the recipe holds across both, but the verification PDF is the one component that must scale differently.

5. LLM workflow with the MCP tool for Rocq

The protocol is the same in both case studies. The LLM produces the Rocq library *autonomously*, driving Rocq through the MCP tool for tactic-level feedback; the blueprint and verification PDF are *not* inputs to this loop: the LLM does not need them to close goals. They are generated *after the fact*, as renderings of the finished library that let a mathematician browse and audit the result. The artifacts are outputs for the mathematician.

Five upstream features. Driven by the two case studies and by feedback from the Rocq-MCP user base, we contributed five features upstream to the MCP server (The LLM4Rocq team, 2026c): stateless goal/query tools at arbitrary file positions, rich proof-state capture on compile failures, per-step feedback with output capping against evaluation blow-ups, workspace auto-detection for multi-theory projects, and position-based proving with process restart. Each is detailed in Appendix D.

6. Discussion

The artifacts[1] let a non-Rocq-expert (i) *check the right thing was formalized*, (ii) *read proofs in mathcomp style*, and (iii) *navigate the dependency graph*. None certify the proofs themselves (Rocq does that), only that *the human read the right statement*.

What to build next. The mathcomp style + blueprint combination scales arbitrarily; the verification PDF is the piece we expect to evolve fastest. As the contrast above shows, the flat, hand-narrated PDF that works for Library E becomes busywork at Library Q's scale, so the right form is a richer, searchable, hyperlinked interface to the same content. The PDF we ship is a first step in that direction.

[1]All artifacts are publicly released: the two libraries (The LLM4Rocq team, 2026a;b), the coding-style skill (The LLM4Rocq team, 2026d), the patched MCP server (The LLM4Rocq team, 2026c), and the Putnam A5 proof (Baudart & Lelarge, 2026).

Acknowledgements

This work was partially supported by the Défi Inria LLM4Code, the Groupe Casino/ENS Chair on Algorithmics and Machine Learning, and the Renaissance Philanthropy project "Leaning and Rocq'ing".

References

Anthropic. Model context protocol, 2024. https://modelcontextprotocol.io.

Axiom Math. Putnam 2025 in Lean, 2025. https://github.com/AxiomMath/putnam2025.

Baudart, G. and Lelarge, M. Putnam 2025 A5 in Rocq, on top of mathcomp-eulerian, 2026. https://github.com/LLM4Rocq/Putnam2025-Rocq.

Baudart, G., Lelarge, M., Stérin, T., and Viennot, J. Putnam 2025 Problems in Rocq using Opus 4.6 and Rocq-MCP, 2026. https://arxiv.org/abs/2603.20405.

Cirpons, R. rocqblueprint: a plasTeX plugin to build Rocq-based formalization blueprints, 2024. https://github.com/reiniscirpons/rocqblueprint.

Google DeepMind. AI achieves silver-medal standard solving International Mathematical Olympiad problems, 2024. AlphaProof and AlphaGeometry 2, IMO 2024.

Heunen, C., Kammar, O., Staton, S., and Yang, H. A convenient category for higher-order probability theory. In *Proceedings of the 32nd Annual ACM/IEEE Symposium on Logic in Computer Science (LICS)*, pp. 1–12, 2017.

Hirata, M., Minamide, Y., and Sato, T. Quasi-Borel Spaces. Archive of Formal Proofs, 2021. https://www.isa-afp.org/entries/Quasi_Borel_Spaces.html.

Massot, P. Lean blueprint, 2024. Plugin and infrastructure used by Liquid Tensor Experiment, Sphere Eversion, Fermat's Last Theorem, and many others.

Mathematical Association of America. The 86th William Lowell Putnam Mathematical Competition, Official Solutions, 2026. https://maa.org/wp-content/uploads/2026/02/2025OfficialSolutions.pdf.

Niven, I. A combinatorial problem of finite sequences. *Nieuw Archief voor Wiskunde*, 16:116–123, 1968.

Stanley, R. P. *Enumerative Combinatorics, Volume 1*, volume 49 of *Cambridge Studies in Advanced Mathematics*. Cambridge University Press, 2 edition, 2011.

The LLM4Rocq team. mathcomp-eulerian: a Rocq formalization of Stanley's Enumerative Combinatorics §1.4 + §1.6, 2026a. https://github.com/LLM4Rocq/mathcomp-eulerian.

The LLM4Rocq team. mathcomp-qbs: a Rocq formalization of Quasi-Borel Spaces, 2026b. https://github.com/LLM4Rocq/mathcomp-qbs.

The LLM4Rocq team. rocq-mcp: an MCP server for the Rocq prover, 2026c. https://github.com/LLM4Rocq/rocq-mcp.

The LLM4Rocq team. mathcomp-skills: a mathcomp coding-style guide and skill, 2026d. https://github.com/LLM4Rocq/mathcomp-skills.

The MathComp-Analysis team. Mathematical components / analysis, 2024. https://github.com/math-comp/analysis.

The Mathematical Components team. Mathematical Components, 2020. https://math-comp.github.io/.

A. Putnam 2025 A5 via Stanley §1.4 + §1.6

This appendix expands the diagnostic vignette of §4.

Problem (Putnam 2025 A5). Let $n \geq 2$. For $s \in \{\pm 1\}^{n-1}$, let $f(s)$ count the permutations $(a_1, \ldots, a_n)$ of $(1, 2, \ldots, n)$ satisfying $s_i\,(a_{i+1} - a_i) > 0$ for all $i = 1, \ldots, n-1$. Determine the sequences s for which $f(s)$ is maximal.

Answer. The two alternating sequences $s_i = (-1)^i$ and $s_i = (-1)^{i+1}$; in both cases $f(s) = E_n$, the n-th Euler / zigzag number (Stanley, 2011; Mathematical Association of America, 2026).

Translation to Stanley §1.4. The descent set of $w \in S_n$ is $D(w) := \{i : w_i > w_{i+1}\}$, and $\beta_n(S) := \#\{w \in S_n : D(w) = S\}$. For a sign sequence s, the constraint $s_i(a_{i+1} - a_i) > 0$ says $a_i > a_{i+1} \iff i \in D_s$ where $D_s = \{i : s_i = -1\}$. Hence $f(s) = \beta_n(D_s)$; the Putnam question is to maximize β_n over $D \subseteq [n-1]$.

Symmetry. The involution $(a_1, \ldots, a_n) \mapsto (n+1-a_1, \ldots, n+1-a_n)$ shows $\beta_n(D) = \beta_n([n-1] \setminus D)$, so the two alternating sets $D_+ = \{1, 3, 5, \ldots\} \cap [n-1]$ and $D_- = \{2, 4, 6, \ldots\} \cap [n-1]$ yield the same value.

Identification with the Euler number. Stanley (Stanley, 2011), §1.4, identifies D_+ and D_- as the descent sets of alternating and reverse alternating permutations; §1.6 gives $\sum_{n \geq 0} E_n x^n/n! = \sec x + \tan x$, an identity now formalized in Library E as `stanley_1_6_1`, an equality of formal power series over $\mathbb{Q}$, via the `mathcomp_fps` sub-library.

Strict inequality. The remaining step, not contained in §1.4 / §1.6, is a Niven-style swap argument (Niven, 1968): if D contains two consecutive elements $k, k+1$, an injection from permutations with descent set D into those with descent set $D' := D \triangle \{k+1\}$ that is not surjective gives $\beta_n(D) < \beta_n(D')$. Iterating leaves only the alternating sets.

Why this matters for the paper. Once Library E is available, every non-Niven step of the above reduces to a single library call. The full A5 proof on top of Library E is 214 lines: 56 lines, problem statement; 102 lines, bridge from the boolean-tuple `s_compat` predicate to the library's set-valued `descent_set`; 56 lines, final theorem composing the bridge with `beta_alt_max` from `mathcomp_eulerian.beta_swap`. The headline statement `putnam_2025_a5` is axiom-free (`Print Assumptions` reports *"Closed under the global context"*). The corresponding bespoke Lean development in (Axiom Math, 2025) weighs in at 2,054 lines, roughly an order of magnitude longer.

B. One page of Library E's verification PDF

The page below is reproduced verbatim from Library E's verification PDF (`eulerian_formal.pdf`, page 19, §5.4 *The insert-max bijection*). It illustrates the artifact's standard layout: an informal **Definition**, a [Rocq, file:line] reference, the Rocq source block, a Decoded. mechanical translation, and, where useful, a shaded *Design choice* sidebar explaining the mathcomp idiom (here, why `match unlift p i with` is the universal case-split for inserting a position into `'I_n.+2`).

5.4 The insert-max bijection

A central tool for the Eulerian recurrence is the bijection that inserts the maximum value $n+1$ at a chosen position into a permutation of $\{1,\dots,n\}$.

Definition 5.13 (Insert-max)**.** For $t \in \mathfrak{S}_n$ and $p \in \{1,\dots,n+1\}$, let `insert_max`$(t,p) \in \mathfrak{S}_{n+1}$ be the permutation obtained by inserting the value $n+1$ at position p in the one-line form of t.

[Rocq, eulerian.v:134-157]

```
Definition insert_max_fun (i : 'I_n.+2) : 'I_n.+2 :=
  match unlift p i with
  | Some j ⇒ widen_ord (leqnSn _) (t j)
  | None ⇒ ord_max
  end.

Definition insert_max_perm : {perm 'I_n.+2} := perm
    insert_max_fun_inj.
```

Decoded. For $i \in \{0,\dots,n+1\}$: if $i = p$, output $n+1$ (the new max); otherwise, write $i =$ `lift p j` for the unique $j \in \{0,\dots,n\}$ that skips p, and output $t(j)$ cast into the larger type.

Design choice: why `match unlift p i with`?

The fundamental case-split for inserting at position p is "is this position p or not?". Mathcomp encodes this as the `option`-valued partial inverse `unlift p`: it returns `None` exactly when $i = p$, and `Some j` otherwise — where $j : 'I_{n+1}$ is the unique witness of $i =$ `lift p j`. Carrying j in the `Some` branch (rather than recomputing it from i and p) avoids any arithmetic juggling: we get the correct `'I_n.+1` value with its bound proof for free, and can apply t directly. This is the universal idiom for constructing permutations on `'I_n.+2` from a permutation on `'I_n.+1` plus a special index.

Definition 5.14 (Extract-max)**.** The inverse: given $\sigma \in \mathfrak{S}_{n+1}$ with $\sigma(p) = n+1$ for some position p, remove that value and standardise to obtain $t \in \mathfrak{S}_n$.

[Rocq, eulerian.v:185-203]

```
Definition extract_max_fun (j : 'I_n.+1) : 'I_n.+1 :=
  odflt j (unlift ord_max (s (lift p j))).

Definition extract_max_perm : {perm 'I_n.+1} := perm
    extract_max_fun_inj.
```

Decoded. For $j \in \{0,\dots,n\}$: look at s at position j of the original (skipping p); the result is some value in $\{0,\dots,n+1\}$ that is not $n+1$ (because $\sigma(p) = n+1$ and σ is a bijection); strip the high bit to land in $\{0,\dots,n\}$. The `odflt j (...)` provides a default that is provably unreachable.

Theorem 5.15 (Insert-max is a bijection)**.** *The map* $(t,p) \mapsto$ `insert_max`(t,p) *is a bijection* $\mathfrak{S}_n \times \{1,\dots,n+1\} \to \mathfrak{S}_{n+1}$.

[Rocq, eulerian.v:437]

```
Lemma insert_max_perm_bij n :
  ∀ sigma : {perm 'I_n.+2},
    ∃! tp : {perm 'I_n.+1} * 'I_n.+2,
      sigma = insert_max_perm tp.1 tp.2.
```

Decoded. Every $\sigma \in \mathfrak{S}_{n+2}$ arises uniquely as `insert_max`(t,p) for some $(t,p) \in \mathfrak{S}_{n+1} \times \{0,\dots,n+1\}$.

Proof sketch. Given σ, the unique p is $\sigma^{-1}(n+1)$ and $t =$ `extract_max`(σ,p). The two roundtrips are direct calculations. □

The page above sets up the insert-max construction. The **lemma that uses it**, the bijection $\{\text{perm } 'I_n. + 1\} \times 'I_n. + 2 \cong \{\text{perm } 'I_n. + 2\}$ on which the Eulerian recurrence rests, appears a few sections later in `eulerian.v:446`, with its proof script reproduced below.

```
(** The ascent count [n - des t] equals the sum of ascent indicators
    over positions. *)
(* ------------------------------------------------------------------- *)
(* Main recurrence                                                     *)
(* ------------------------------------------------------------------- *)

(** Proof-irrelevant variant of [extract_insert_max], threading any
    equality proof [insert_max_perm t p p = ord_max] rather than the
    canonical one. *)
(** The pairing map [(t, p) |→ insert_max_perm t p] is a bijection
    [{perm 'I_n.+1} x 'I_n.+2] ~= [{perm 'I_n.+2}]. Foundation of the
    recurrence. *)
Lemma insert_max_perm_bij n :
  bijective (fun tp : {perm 'I_n.+1} * 'I_n.+2 ⇒
             insert_max_perm tp.1 tp.2).
Proof.
pose g (s : {perm 'I_n.+2}) : {perm 'I_n.+1} * 'I_n.+2 :=
  (extract_max_perm (permKV s ord_max), (s^-1)%g ord_max).
exists g.
- move⇒ [t p] /=; rewrite /g /=.
  (* First extract p *)
  congr pair; last exact: insert_max_perm_fiber.
  apply/permP ⇒ j; apply: (@widenSn_inj n).
  rewrite extract_max_widen.
  have E : lift ((insert_max_perm t p)^-1%g ord_max) j = lift p j
                                                      :> 'I_n.+2.
    by apply/val_inj; rewrite /= insert_max_perm_fiber.
  by rewrite E insert_max_perm_lift.
- move⇒ s /=; rewrite /g; exact: insert_extract_max.
Qed.
```

Read alongside the previous page: the informal **Definition 5.13** (insert-max) and **Definition 5.14** (extract-max) are the two halves of the bijection `g` above; the inverse-laws `insert_max_perm_fiber`, `insert_max_perm_lift`, `insert_extract_max`, `extract_max_widen` are the supporting lemmas that the verification PDF surfaces in subsequent pages.

C. The coding-style skill

The coding-style skill (The LLM4Rocq team, 2026d) (§3(a)) comprises: ≈37 sections of core conventions (line length, `mainSymbol_suffix` naming, proof style, HB instances, deprecation, idiomatic SSReflect, search discipline); domain-specific idiom files (`matrix`, `polynomial`, `finset`, `int`/`rat`, `derive`, `measure`, `topology`, algebra-tactics, `tuple`/`perm`/`binomial`, `finfun`); a verified playbook of cleanup patterns; date-stamped end-to-end verification stamps against specific Rocq / mathcomp versions; and a mechanical POSIX shell audit script for high-yield style violations.

D. MCP server features

The five features contributed upstream to the Rocq-MCP server (The LLM4Rocq team, 2026c) (§5):

(i) **stateless goal + query tools at arbitrary file positions** (cheap "look before you leap" without opening a session);

(ii) **rich proof-state capture on compile failures**: the response now ships open goals and hypotheses at the failing sentence, reconstructed via `coq-lsp`;

(iii) **per-step feedback in `rocq_check` / `rocq_step_multi`**, with output capping against `vm_compute` / `native_compute` token explosions;

(iv) **workspace auto-detection, Dune support, multi-theory workspaces**, removing a class of "Theorem not found" failures on project-local imports;

(v) **position-based proving + process restart**: capture proof state at arbitrary positions as named sessions (for Section-local lemmas), restart `pet` / `coq-lsp` to invalidate stale `.vo` caches.

E. Library Q limitations in detail

The probability monad is built with a weaker random-element condition, with the monad laws stated up to equivalence. The $L \dashv R$ correspondence is a hom-set bijection rather than a full natural transformation. Iterated integration and a factorization lemma are stated on a specific QBS product measure rather than as general theorems. Standard Borel spaces are characterized constructively via an encode/decode pair, as in Isabelle, rather than via Polish spaces as in Lean. A few expectation results carry integrability hypotheses that are not yet in mathcomp-analysis. All are listed in the blueprint, so a reader auditing the library can see at a glance what was and was not formalized.